

Projeto Threads - Alarm Clock

Participantes:

Pedro Arthur Santana Patriota

Yasmim Vitoria Silva de Oliveira

Pedro Augusto de Almada Lobo Fonseca

1. Decisões de implementação (*design*)

Estrutura de Dados

pintos/src/threads/thread.c

```
static struct list sleep_list;
```

Lista responsável por armazenar as threads bloqueadas pelo estado **sleep**. As threads são armazenadas em ordem crescente de acordo com *tick* que elas devem acordar.

Algoritmos

A lógica do novo **timer_sleep()** funciona a partir do cálculo do valor de *ticks* que teremos após o tempo de espera determinado, ou seja, quando a thread deveria acordar.

pintos/src/devices/timer.c

```
/* Sleeps for approximately TICKS timer ticks. Interrupts must
   be turned on. */
void
timer_sleep (int64_t ticks)
{
    int64_t wakeup_tick = timer_ticks () + ticks;
    enum intr_level old_level = intr_disable ();
    thread_sleep (wakeup_tick);
    intr_set_level (old_level);
}
```

Com esse valor em mãos, nós fazemos a inserção da thread que será pausada na lista de **sleep_list**, mantendo sempre a ordem crescente de quando cada thread deve acordar.

pintos/src/threads/thread.c

```
void
thread_sleep (int64_t wakeup_tick)
{
```

```

struct thread *cur_thread = thread_current ();
cur_thread->wakeup_tick = wakeup_tick;
list_insert_ordered (&sleep_list, &cur_thread->sleepelem, wakeup_tick_less,
                    NULL);
thread_block ();
}

```

Uma vez que inserimos a thread atual na **sleep_list**. Quem é responsável por verificar o momento de acordar as threads bloqueadas é o próprio **timer_interrupt()**, responsável por fazer a contagem de ticks e ser o interrupt handler do timer.

pintos/src/devices/timer.c

```

/* Timer interrupt handler. */
static void
timer_interrupt (struct intr_frame *args UNUSED)
{
    ticks++;
    thread_tick ();

    int64_t now = timer_ticks ();
    thread_wake (now);
}

```

Assim, em cada frame, nós checamos a **sleep_list** e vemos se no topo da fila a thread que deve acordar mais cedo está apta a ser acordada. Se sim, a tiramos da **sleep_list** e a colocamos disponível para execução, com isso, seguimos comparando os próximos topos da lista. Se não, mantemos a **sleep_list** sem alteração.

pintos/src/threads/thread.c

```

void
thread_wake (int64_t now)
{
    struct thread *cur_thread = NULL;
    while (!list_empty (&sleep_list))
    {
        cur_thread = list_entry (list_front (&sleep_list), struct thread,
                                sleepelem);
        if (cur_thread->wakeup_tick > now)
            break;
        list_pop_front (&sleep_list);
        thread_unblock (cur_thread);
    }
}

```

Por causa disso, o **busy waiting** é evitado e fazemos uma checagem em tempo constante a cada ciclo para checar a possibilidade de desbloquearmos a threads em *sleep*.

Sincronização

Nessa parte do projeto, escolhemos por desabilitar *interrupts* (**timer_interrupt**) durante as operações na **sleep_list** no kernel de threads, isso porque *locks* não podem ser aplicados aos interrupt handlers.

Desse modo, garantimos que a inserção na **sleep_list** não é afetada pela concorrência no momento da sua deleção pelo **timer_interrupt**, garantindo a ordem correta de threads baseados no momento que devem ser acordadas.

2. Testes

```
pass tests/threads/alarm-single
pass tests/threads/alarm-multiple
pass tests/threads/alarm-simultaneous
FAIL tests/threads/alarm-priority
pass tests/threads/alarm-zero
pass tests/threads/alarm-negative
FAIL tests/threads/priority-change
FAIL tests/threads/priority-donate-one
FAIL tests/threads/priority-donate-multiple
FAIL tests/threads/priority-donate-multiple2
FAIL tests/threads/priority-donate-nest
FAIL tests/threads/priority-donate-sema
FAIL tests/threads/priority-donate-lower
FAIL tests/threads/priority-fifo
FAIL tests/threads/priority-preempt
FAIL tests/threads/priority-sema
FAIL tests/threads/priority-condvar
FAIL tests/threads/priority-donate-chain
FAIL tests/threads/mlfqs-load-1
FAIL tests/threads/mlfqs-load-60
FAIL tests/threads/mlfqs-load-avg
FAIL tests/threads/mlfqs-recent-1
pass tests/threads/mlfqs-fair-2
pass tests/threads/mlfqs-fair-20
FAIL tests/threads/mlfqs-nice-2
FAIL tests/threads/mlfqs-nice-10
FAIL tests/threads/mlfqs-block
```

3. Arquivos alterados ou adicionados

pintos/src/devices/timer.c

pintos/src/threads/thread.c
pintos/src/threads/thread.h